

Database: Create & Use

- `use <databaseName>` switches (and selects) the current database context.
- MongoDB has no dedicated command to create a database ahead of time — switching to a name that doesn't exist yet selects it, but the database is only created once you insert data into a collection within it.
- Every command that follows `use` runs against the newly selected database.

```
use schoolDB
// The database is now selected, but doesn't exist on disk
// until a collection is written to it.
db.students.insertOne({ name: "Aditi", age: 18 });
// schoolDB now exists.
```

Database: List (show dbs)

- **show dbs** lists every database on the connected MongoDB server, along with its storage size.
- A database only appears in this list once it actually holds data — matching the earlier point that use alone doesn't create it.

```
show dbs
```

```
// admin      0.000GB
```

```
// config     0.000GB
```

```
// local      0.000GB
```

```
// schoolDB   0.000GB
```

Database: Delete

- `db.dropDatabase()` permanently deletes the currently selected database, including all of its collections and documents.
- This action cannot be undone.



```
use schoolDB
db.dropDatabase();
// schoolDB and all its data are permanently removed
```

Collection: Create

- `db.createCollection(name, options)` explicitly creates a new collection. Optional in most cases — MongoDB auto-creates a collection on first insert — but useful for setting validation rules or a capped size.



```
use schoolDB
db.createCollection("students");
// Creates an empty "students" collection
```

Collection: List (show collections)

- **show collections** lists every collection that exists in the currently selected database.
- Like databases, a collection only shows up here once it has been created — explicitly, or implicitly via a first insert.



```
use schoolDB
show collections
// students
```

Collection: Delete

- `db.<collection>.drop()` permanently removes a collection, its documents, and its indexes.
- This action cannot be undone.



```
db.students.drop();  
// The "students" collection and all its documents are removed
```

CRUD: Create

- `insertOne(document)` inserts a single document into a collection.
- `insertMany([doc1, doc2, ...])` inserts multiple documents in a single call, passed as an array.
- If the collection doesn't exist yet, MongoDB creates it automatically.

```
db.students.insertOne({
  name: "Rahul", age: 19
});

db.students.insertMany([
  { name: "Meera", age: 17 },
  { name: "Kabir", age: 18 }
]);
```

CRUD: Read

- **find(query, projection)** retrieves all documents matching a filter, returning a cursor. No arguments returns every document.
- **findOne(query, projection)** retrieves just the first matching document (or null), instead of a cursor.



```
db.students.find({ age: 18 });  
// Returns all documents where age equals 18  
  
db.students.findOne({ name: "Rahul" });  
// Returns the first match, or null
```

CRUD: Update

- `updateOne(filter, update)` updates the first matching document, using operators like `$set` to modify specific fields.
- `updateMany(filter, update)` applies the same update to every matching document.

```
db.students.updateOne(  
  { name: "Rahul" },  
  { $set: { age: 20 } }  
);  
  
db.students.updateMany(  
  { age: 18 },  
  { $set: { promoted: true } }  
);
```

CRUD: Delete

- **deleteOne(filter)** deletes the first document that matches a query filter.
- **deleteMany(filter)** deletes every matching document. An empty filter ({}) deletes all documents without dropping the collection.



```
db.students.deleteOne({ name: "Kabir" });  
  
db.students.deleteMany({ age: 17 });  
// Removes every document where age equals 17
```

More About find(): Filter & Projection

- **Query filter** narrows results to documents matching field values or operators (e.g. \$gt, \$in). An empty filter ({}) matches everything.
- **Projection** (2nd argument) controls which fields are returned — 1 to include, 0 to exclude.

```
db.students.find({ age: { $gte: 18 } });  
// Returns students with age 18 or higher  
  
db.students.find({}, { name: 1, age: 1, _id: 0 });  
// Returns only name and age for every student
```

More About find(): sort, limit, skip

- `sort({ field: 1 | -1 })` orders results — 1 ascending, -1 descending.
- `limit(n)` caps the number of documents returned; `skip(n)` omits the first n — commonly combined for pagination.



```
db.students.find().sort({ age: -1 });
```

```
// Highest to lowest age
```

```
db.students.find().sort({ age: 1 }).skip(10).limit(5);
```

```
// Skips the first 10 results, returns the next 5
```

findOne() Revisited

- **findOne()** behaves like **find()** but always returns a single document (the first match) rather than a cursor.
- It does not support cursor methods like **sort()** or **limit()** directly.

```
db.students.findOne(  
  { age: { $gte: 18 } },  
  { name: 1, _id: 0 }  
);  
// Returns the name of the first matching student found
```

Query & Update Operators

Operators used inside query filters and update documents to express conditions and modifications.

Comparison & Array

- \$gt** → Greater than
- \$gte** → Greater than or equal
- \$lt** → Less than
- \$lte** → Less than or equal
- \$ne** → Not equal
- \$in** → Matches values in array
- \$nin** → Not in array

Logical

- \$or** → OR condition
- \$and** → AND condition

Update

- \$set** → Update / set field
- \$inc** → Increment
- \$push** → Add to array
- \$pull** → Remove from array

Query Operators in Action

- **Comparison operators** go inside a field's filter value; **\$or** / **\$and** take an array of condition objects.

```
// age strictly between 16 and 10
db.students.find({ age: { $gt: 16, $lt: 10 } });

// age is 17, 18, or 19
db.students.find({ age: { $in: [17, 18, 19] } });

// age >= 19 OR name is "Rahul"
db.students.find({
  $or: [{ age: { $gte: 19 } }, { name: "Rahul" }]
});
```

Update Operators in Action

- **\$set** replaces a field's value; **\$inc** adds to a numeric field; **\$push/\$pull** add to or remove from an array field.

```
db.students.updateOne(  
  { name: "Rahul" },  
  {  
    $set: { age: 20 },  
    $inc: { attempts: 1 },  
    $push: { subjects: "History" }  
  }  
);  
// Remove "Science" from the subjects array  
db.students.updateOne({ name: "Rahul" }, { $pull: { subjects: "Science" } });
```